FAST — NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES

AL-2002 ARTIFICIAL INTELLIGENCE LAB

PROJECT PROPOSAL · FALL 2026

Smart Warehouse Robot Simulator

Comparing AI Agents and Pathfinding Algorithms

Project Idea No. 15 · Part B — Applied / Data · Medium Difficulty

Group Information

Role	Name	Roll Number	Section	Role in Project
Leader	Anas Haider	24K-2002	BSCY-5A	Environment, integration and repository
Member 2	Shayan Hassan	24K-2050	BSCY-5A	Search algorithms, agents and experiments

GitHub Repository URL: [https://github.com/anashaider/SmartWarehouseAI.git]

Date of Submission: 10/20/2026

1. Project Title and Summary

The Smart Warehouse Robot Simulator is a Python application that simulates robots collecting parcels from a warehouse and delivering them to designated drop-off points. The warehouse is represented as a grid containing shelves, open paths, parcel locations and delivery stations. The project will implement Simple Reflex, Goal-Based and Utility-Based agents and compare their behaviour in the same environment. It will also implement Breadth-First Search (BFS) and A* search for robot path planning. A visual interface will show robot movement, completed deliveries and travelled distance, while experiments will compare the agents and search methods using consistent test layouts and fixed random seeds.

2. Problem Statement and Motivation

Warehouse robots must navigate around shelves, reach parcel locations and deliver items without colliding with obstacles or other robots. Even in a small grid, the number of possible routes and action sequences can grow quickly as the warehouse becomes larger or more obstacles are added. A robot that simply moves toward a parcel may choose an unhelpful direction, take a longer route or become blocked by another robot. Multiple robots make the problem more difficult because their routes and occupied cells can interfere with one another. In a real warehouse, inefficient movement can increase delivery time, energy use and congestion. This project studies the problem in a controlled simulation, where layouts and task sequences can be repeated fairly. By comparing agent designs and pathfinding algorithms, the system will demonstrate how goal-directed decisions and informed search can improve robot navigation. The simulator is worthwhile because it makes AI agent behaviour observable and measures performance rather than relying only on visual demonstrations.

3. Problem Formulation

3.1 PEAS Description

Component	Description
Performance measure	Parcels delivered, total path length, task completion time, invalid moves and robot conflicts.
Environment	Discrete warehouse grid with shelves, free paths, parcel pickup points and drop-off stations; two or three robots may operate together.
Actuators	Move up, down, left or right; pick up a parcel; deliver a parcel; wait.
Sensors	Current position, nearby obstacles, assigned parcel, destination, parcel status and other robots' occupied cells.

3.2 Environment and Agent Type

Property	Classification
Observability	Fully observable in the simulator
Determinism	Deterministic for a fixed layout, task list and random seed
Change over time	Dynamic
State and actions	Discrete

Agents	Multi-agent when two or three robots operate together
Agent type	Simple Reflex, Goal-Based and Utility-Based

3.3 Problem Representation

Element	Representation
State representation	Grid cells, robot positions, carried parcels, remaining parcels, delivery stations and occupied cells.
Actions / moves	Move up/down/left/right, pick up, deliver or wait, subject to valid-move rules.
Initial state	A predefined or seeded-generated layout with robots, parcels and drop-off points placed in valid cells.
Goal or terminal test	All assigned parcels have been delivered, or a defined step limit is reached.
Cost / evaluation	Each movement has unit cost; total path length and completion time are measured. The utility agent balances distance, progress and conflict avoidance.

4. Proposed AI Approach

#	Technique	Role in Project	Implementation
1	Breadth-First Search (BFS)	Finds a shortest path on an unweighted grid; uninformed-search comparison.	Own Python code
2	A* Search	Finds a path using movement cost plus a Manhattan-distance heuristic.	Own Python code
3	Agent strategies	Simple Reflex uses rules; Goal-Based plans toward a target; Utility-Based scores progress, distance and conflict avoidance.	Own Python code
B	Baseline	Fixed-order or nearest-target strategy evaluated on the same layouts and tasks.	Own Python code

BFS and A* will report path cost, nodes expanded and runtime. The Manhattan heuristic is the sum of horizontal and vertical distance to the goal. With four-direction movement and unit step costs, it does not overestimate the remaining path cost. The simulator will prevent moves into shelves or cells occupied by another robot and handle blocked moves safely.

4.1 Why These Techniques Suit the Problem

The warehouse is a discrete grid, so search algorithms can represent locations as states and valid moves as actions. BFS explores paths in increasing order of depth and finds a shortest path when all moves have equal cost. A* combines movement cost with Manhattan distance to prioritise states that appear closer to the goal. The three agent types demonstrate different decision-making approaches: a Simple Reflex Agent uses immediate rules, a Goal-Based Agent acts to reach a target, and a Utility-Based Agent evaluates choices using a score. The utility score will favour delivery progress while penalising unnecessary distance and conflicts. All methods will be tested on identical seeded layouts for reproducibility.

4.2 Planned Class Design

- WarehouseEnvironment: stores the grid, shelves, parcels, drop-off points and robots; provides is_valid_move(), get_percept(), apply_action() and is_complete().
- RobotAgent (base class): stores robot ID, position, assigned parcel and carrying status; provides choose_action() and update_state().
- SimpleReflexAgent, GoalBasedAgent and UtilityBasedAgent: inherit from RobotAgent and override choose_action() with their respective strategies.
- Pathfinder: implements bfs(start, goal), astar(start, goal) and path reconstruction.
- ExperimentRunner: runs seeded scenarios and records delivery count, distance, runtime and nodes expanded.

5. Scope

In Scope

#	Feature	Why Essential
1	Grid warehouse with shelves, parcels and drop-off points	Defines the environment and tasks.
2	Simple Reflex, Goal-Based and Utility-Based agents	Demonstrates the required agent types.
3	Own implementations of BFS and A*	Provides the core AI search techniques.
4	Two or three robots with basic conflict avoidance	Demonstrates multi-agent operation.
5	Dashboard and repeatable experiments with at least three charts	Makes performance measurable and comparable.

Out of scope: The project will not control physical robots, connect to warehouse hardware, use paid APIs, or build a production-grade warehouse management system. Advanced reinforcement learning, dynamic real-world mapping and complex collision coordination beyond basic cell-conflict handling are excluded. The focus is a reliable educational simulation.

6. Data or Environment Source

Field	Description
Source name and URL	Generated / simulated warehouse layouts; no external dataset or URL is required.
Type	Generated / simulated
Access requirements	No key needed; no paid source.
Approximate size	At least 20 seeded scenarios with varied shelf layouts and parcel positions.
Licence / terms of use	The team will generate the layouts and task instances itself.
Offline fallback	Layouts and tasks will be generated locally or stored in local files; the simulator and experiments will run without a network connection.

7. Evaluation Plan

Item	Plan
What will be compared	BFS vs. A* pathfinding; Simple Reflex, Goal-Based and Utility-Based agent performance; fixed-order baseline.
Metrics	Parcels delivered, total distance, completion steps, runtime, nodes expanded, invalid moves and robot conflicts.
Baseline	Fixed-order / nearest-target strategy and BFS where appropriate.
Test instances / test set	At least 20 generated warehouse scenarios with varied shelf layouts and parcel placements.
Number of runs and seed	Use fixed seeds; all methods use the same scenarios. Repeat runs where stochastic choices are introduced.
Planned charts (at least three)	(1) Average path length by algorithm; (2) nodes expanded by BFS and A*; (3) parcels delivered by agent type; (4) runtime comparison if useful.

The experiment runner will record the seed, scenario identifier and metrics in a results file. Charts and summary tables will be generated from actual recorded results; no performance figures will be claimed before the experiments are run.

8. Division of Work

Member	Modules / components owned	Approximate share
Anas Haider (24K-2002) — Leader	Warehouse environment, integration, interface, repository and README.	50%
Shayan Hassan (24K-2050) — Member 2	BFS/A*, agent strategies, experiment runner and result charts.	50%

The division is provisional and should be updated to match actual work and GitHub commits.

9. Milestones and Timeline

#	Demonstrable milestone	Target date	Owner
1	Warehouse grid loads, validates moves and places robots, parcels and drop-off points.	Week 1 after approval	Anas
2	BFS and A* return valid paths and report path cost and nodes expanded.	Week 2	Shayan
3	Three agent types operate; basic multi-robot conflict handling works.	Week 3	Both
4	Seeded experiment runner produces a results table and at least three labelled charts.	Week 4	Shayan
5	Final integration, tests, README, PDF/MD documentation and clean-clone run complete.	By Week 15 deadline	Both

Dates are relative to project approval; replace them with calendar dates once the instructor announces the submission deadline.

10. Expected Outcome and Risks

The completed project will open a visual warehouse grid in which robots travel between parcel locations and drop-off stations while avoiding shelves and occupied cells. Users will observe agent decisions and run repeatable scenarios. The experiment results will compare BFS and A* pathfinding effort and agent strategies on delivery count, distance and completion steps. One risk is that multiple robots may repeatedly block each other; the fallback is safe waiting and basic turn ordering, with remaining limitations documented. A second risk is that the visual interface may take longer than expected; the fallback is a working console-based simulation and experiment runner, with the interface added around the completed AI logic.

Before Submission

Fill in the actual section, GitHub repository URL and submission date. Confirm that Idea No. 15 is available on the section group sheet. Update this proposal if the instructor requests changes and ensure the final report describes what is actually implemented.